

0.95.1. Un micro Zelda — l'écran glisse dans le sens du A

Le 0.95, et à chaque passage d'une salle à une autre, l'écran GLISSE dans le sens où va le A : de gauche à droite, de droite à gauche, de haut en bas, de bas en haut. Avec `defiler(x, y)`, 2 pixels par image.

1. Ce qu'il faut comprendre

C'est le 0.95, avec une chose en plus : le glissement. Quand le A passe d'une salle à une autre, la nouvelle salle n'apparaît plus d'un coup : l'écran **glisse**, comme dans Zelda, **dans le sens où va le A**. Il va à **gauche** : l'ancienne salle part vers la droite, la nouvelle arrive par la gauche. À **droite** : l'ancienne part vers la gauche, la nouvelle arrive par la droite. En **haut** : l'ancienne descend, la nouvelle arrive par le haut. En **bas** : l'ancienne monte, la nouvelle arrive par le bas. Les **portes**, elles, téléportent : d'un coup, comme au 0.95.

Ce qui vient d'avant : tout le 0.95 (les quatre salles, `sx`, `sy`, les murs, les portes, le 255 d'un `uint8_t` qui fait le tour), et `defiler(x, y)` du 0.90.3 : il ne bouge pas une lettre, il déplace **la caméra** qui regarde le décor, au pixel près. `x` la pousse vers la droite, `y` vers le bas.

Ce qui est nouveau ici (1) : le monde fait le tour. Chaque salle a maintenant **quatre sorties** : à gauche, à droite, en haut, en bas. À droite de la salle 1, on revient à la salle 0 ; à gauche de la salle 0, on arrive à la salle 1 ; en haut de la salle 0, on arrive à la salle 2. Ainsi, **chaque** sortie mène à une salle, et l'écran glisse à chaque fois, dès le départ. Avec deux colonnes de salles, changer de colonne s'écrit `sx = 1 - sx` : $1 - 0 = 1$, $1 - 1 = 0$. Pareil pour les lignes : `sy = 1 - sy`.

Ce qui est nouveau ici (2) : le décor est plus grand que l'écran. Le décor de la console fait **32 colonnes sur 32 lignes** (256×256 pixels) ; l'écran n'en montre que **20 sur 18** (160×144 pixels). Le reste attend, caché. Et le décor est un **ruban**, dans les deux sens : après la colonne 31 revient la colonne 0, après la ligne 31 revient la ligne 0.

Ce qui est nouveau ici (3) : la salle n'est plus toujours dans le coin (0, 0) du décor. Deux variables le retiennent : `bord`, la colonne du décor où commence la salle, et `haut`, la ligne. Une case `(c, l)` **de la salle** est la case `((c + bord) % 32, (l + haut) % 32)` **du décor**. `% 32`, le reste de la division par 32, fait le tour du ruban : avec `bord = 12`, la colonne 19 de la salle est $(19 + 12) \% 32 = 31$; une colonne 20 serait $(20 + 12) \% 32 = 32 \% 32 = 0$. La caméra suit : `camx = bord × 8`, `camy = haut × 8` (8 pixels par case).

Ce qui est nouveau ici (4) : poserSalle() et lireSalle(). Elles font ce calcul pour nous : `poserSalle(c, l, tuile)` au lieu de `poser()`, `lireSalle(c, l)` au lieu de `lire()`. Tout le programme passe par elles, et `ax`, `ay` restent des cases **de la salle**. `lireSalle()` **rend** un nombre : `uint8_t` devant son nom, et `return` pour dire lequel. Pour effacer, on pose la tuile **0** (une case vide). Et « SALLE n » s'écrit maintenant lettre par lettre avec `poserSalle()` (le chiffre n est la tuile `27 + n`) : `texte()` et `nombre()` ne savent pas ajouter `bord` et `haut`.

Ce qui est nouveau ici (5) : dessiner une seule colonne, ou une seule ligne. Le dessin de la salle (cadre, ouvertures, murs, portes, « SALLE n ») devient la fonction `dessinerMurs()`. Deux variables disent ce qu'elle dessine : `seule` (une colonne) et `seuleL` (une ligne) ; **255** veut dire « toutes ». C'est `poserSalle()` qui trie : `(seule == 255 || c == seule) && (seuleL == 255 || l == seuleL)`. `&&` veut dire « et » : les deux doivent

être vrais. `colonneSalle(c)` vide la colonne `c`, puis la dessine, elle seule ; `ligneSalle(l)` fait pareil pour une ligne.

Ce qui est nouveau ici (6) : les quatre glissements. Quand le A sort à **gauche**, `glisserAGauche()` : la nouvelle salle se met 20 colonnes plus à gauche (`bord + 12`, car $32 - 20 = 12$ sur le ruban). Puis 20 tours : on dessine la colonne qui va entrer par la gauche (19, puis 18... jusqu'à 0), et la caméra **recule** d'une case en **4 petits pas de 2 pixels**, un par image : `camx = camx - 2`, `defiler(camx, camy)`. À **droite**, `glisserADroite()` : 20 colonnes plus à droite (`bord + 20`), on dessine les colonnes 0, 1... 19, et la caméra **avance** : `camx = camx + 2`. En **haut** et en **bas**, c'est pareil, mais debout : ligne par ligne avec `ligneSalle()`, et c'est `camy` qui bouge. La salle a 18 lignes : 18 lignes plus haut, c'est `haut + 14` ($32 - 18 = 14$) ; plus bas, `haut + 18`.

Déroulons, à gauche, depuis le départ (`bord` = 0, caméra à 0). Le A va jusqu'en (0, 8), l'ouverture, puis encore GAUCHE : l'arrivée est $0 - 1 = 255$, sorti à gauche. `sx` devient $1 - 0 = 1$, `bord` devient 12. Tour 0 : la colonne 19 de la salle 1 va dans la colonne $(19 + 12) \% 32 = 31$ du décor, cachée ; la caméra passe à $0 - 2 = 254$ (elle fait le tour), puis 252, 250, 248 : la colonne 31 entre par la gauche, 2 pixels à chaque image. ... Tour 11 : la colonne 8 va dans la colonne **20**, la dernière colonne cachée. Tour 12 : la colonne 7 va dans la colonne **19**. C'était l'ancienne salle ! Mais elle vient de sortir de l'écran par la droite : on peut la remplacer. ... Tour 19 : la colonne 0 va dans la colonne 12 ; la caméra est à **96** (12×8) : l'écran montre les colonnes 12 à 31, toute la salle 1. Le A est posé en (19, 8).

Pourquoi 4 petits pas de 2 pixels ? 8 pixels d'un coup à chaque image, c'est trop rapide : les 20 colonnes passeraient en 20 images, un tiers de seconde, et l'œil ne verrait pas glisser. Avec 2 pixels par image : $20 \times 4 = 80$ images à gauche et à droite, $18 \times 4 = 72$ en haut et en bas, un peu plus d'une seconde.

Pourquoi vider la colonne (ou la ligne) avant de la dessiner ? Au milieu du glissement, elle contient encore l'ancienne salle. `dessinerMurs()` ne pose que des X, des P, des lettres et quelques cases vides : un X de l'ancienne salle, là où la nouvelle n'en a pas, resterait.

Le A pendant le glissement : on le retire avant (sinon, il partirait avec l'ancienne salle), et on le pose après, du côté par où il entre : sorti à gauche, il entre à droite, en colonne 19 ; sorti en haut, il entre en bas, en ligne 16. L'autre coordonnée ne change pas.

Et les portes ? `dessinerSalle()` redessine d'un coup, et remet tout à zéro : `bord` = 0, `haut` = 0, la caméra à (0, 0). `viderEcran()` efface maintenant **tout** le décor, 32 lignes de 32 cases : les parties cachées doivent être vides, car un glissement va les montrer.

Essaie : pour glisser plus vite, fais 2 petits pas de 4 pixels : `p < 2` et `camx = camx - 4` ($2 \times 4 = 8$, toujours une case). Plus lentement : 8 petits pas de 1 pixel. La règle : le nombre de pas $\times$ les pixels par pas = 8.

Les fonctions de cette leçon — et le tuto de chacune

- `#include <poser>` → [0.1.3. La fonction poser\(\) — poser une tuile](#)
- `#include <lire>` → [0.68.4. La fonction lire\(\) — lire une case](#)
- `#include <ALPHABET>` → [0.1.4. ALPHABET — les lettres de la police](#)
- `#include <effacer>` → [0.16.3. La fonction effacer\(\) — effacer des cases](#)
- `#include <defiler>` → [0.90.3. La fonction defiler\(\) — faire glisser le fond](#)
- `#include <chaque>` → [0.29.3. La fonction chaque\(\) — toutes les n millisecondes](#)
- `#include <bouton>` → [0.65.5. La fonction bouton\(\) — lire la manette](#)

2. Le programme, morceau par morceau

Le programme entier de la leçon, découpé comme l'œil le lit. Il fait **2599 octets** de cartouche.

Ce que le compilateur y a gravé, nommé — chaque morceau, sa taille, qui l'a demandé — et ce qu'il a laissé, parce que rien ne l'appelle :

Gravé dans la cartouche :

démarrage	92 o	← la console, à l'allumage – la mise en route de la console
chaque_minuteur()	124 o	← main() – une fonction fournie par le langage
poserSalle()	62 o	← mur(), murDebout(), dessinerMurs(), colonneSalle(), ligneSalle(), main() – ta fonction
lireSalle()	28 o	← main() – ta fonction
mur()	40 o	← dessinerMurs() – ta fonction
murDebout()	40 o	← dessinerMurs() – ta fonction
viderEcran()	33 o	← dessinerSalle() – ta fonction
dessinerMurs()	442 o	← dessinerSalle(), colonneSalle(), ligneSalle() – ta fonction
dessinerSalle()	35 o	← main() – ta fonction
colonneSalle()	44 o	← glisserAGauche(), glisserADroite() – ta fonction
ligneSalle()	44 o	← glisserEnHaut(), glisserEnBas() – ta fonction
glisserAGauche()	81 o	← main() – ta fonction
glisserADroite()	77 o	← main() – ta fonction
glisserEnHaut()	81 o	← main() – ta fonction
glisserEnBas()	77 o	← main() – ta fonction
main()	440 o	← démarrage – ta fonction
VBlank	8 o	← la console, à chaque image – compte les images, 60 fois par seconde
AttendreImage	28 o	← image() – attend l'image suivante
AttendreVBlank	16 o	← démarrage – attend que l'écran soit libre
AttendreAcces	12 o	← EffacerCases – attend de pouvoir écrire une case
AdresseCase	10 o	← poser(), lire(), effacer() – trouve l'adresse d'une case de l'écran
EffacerCases	12 o	← effacer() – efface des cases
CopierTuiles	18 o	← démarrage – copie les lettres et les dessins en mémoire vidéo
EffacerCarte	14 o	← démarrage – vide les deux cartes de l'écran
LireManette	37 o	← bouton() – lit les huit boutons
Tuiles	704 o	← CopierTuiles – la police entière

Lettres gravées : toutes (poser() reçoit un numéro calculé)

Pas gravé (jamais appelé) : AvancerAirs, AttendreFinVBlank, EcrireTexte, EcrireNombre, Diviser, Reste, Multiplier, DecalerGauche, DecalerDroite, RangerLutins, InstallerTransfert, Hasard, EffacerFond, EffacerPanneau, DonneesTransfert, Notes

Fonctions du langage appelées :

defiler()	x 5	40 o sur place, dans dessinerSalle(), glisserAGauche(), glisserADroite(), glisserEnHaut(), glisserEnBas()
poser()	x 1	29 o sur place, dans poserSalle() + AdresseCase (10 o)
lire()	x 1	27 o sur place, dans lireSalle() + AdresseCase (10 o)
bouton()	x 4	20 o sur place, dans main() + LireManette (37 o)
image()	x 5	15 o sur place, dans glisserAGauche(), glisserADroite(), glisserEnHaut(), glisserEnBas(), main() + AttendreImage (28 o)
effacer()	x 1	14 o sur place, dans viderEcran() + AdresseCase (10 o), EffacerCases (12 o)
images()	x 1	2 o sur place, dans chaque_minuteur()

Tes fonctions appelées :

poserSalle()	x 34	497 o sur place, dans mur(), murDebout(), dessinerMurs(), colonneSalle(), ligneSalle(), main()
mur()	x 6	87 o sur place, dans dessinerMurs()
murDebout()	x 4	57 o sur place, dans dessinerMurs()
lireSalle()	x 2	22 o sur place, dans main()
colonneSalle()	x 2	18 o sur place, dans glisserAGauche(), glisserADroite()
ligneSalle()	x 2	18 o sur place, dans glisserEnHaut(), glisserEnBas()
chaque_minuteur()	x 1	10 o sur place, dans main()
dessinerMurs()	x 3	9 o sur place, dans dessinerSalle(), colonneSalle(), ligneSalle()
dessinerSalle()	x 2	6 o sur place, dans main()
viderEcran()	x 1	3 o sur place, dans dessinerSalle()
glisserAGauche()	x 1	3 o sur place, dans main()
glisserADroite()	x 1	3 o sur place, dans main()
glisserEnHaut()	x 1	3 o sur place, dans main()
glisserEnBas()	x 1	3 o sur place, dans main()

1. CE QUE LE PROGRAMME RACONTE DE LUI-MÊME

```
// ---- UN MICRO ZELDA : L'ÉCRAN GLISSE DANS LE SENS DU A ----
// C'est le 0.95, avec une chose en plus : à CHAQUE passage d'une salle
// à une autre, la nouvelle salle n'apparaît plus d'un coup. L'écran
// GLISSE, dans le sens où va le A :
// le A va à GAUCHE : l'ancienne salle part vers la droite, la nouvelle arrive par la gauche ;
// le A va à DROITE : l'ancienne salle part vers la gauche, la nouvelle arrive par la droite ;
// le A va en HAUT : l'ancienne salle part vers le bas, la nouvelle arrive par le haut ;
// le A va en BAS : l'ancienne salle part vers le haut, la nouvelle arrive par le bas.
// Les portes, elles, téléportent : elles redessinent d'un coup, comme au 0.95.
//
// Tout ce qui suit « // » sur une ligne est un COMMENTAIRE : la console
// ne le lit pas. Il n'est là que pour toi, pour expliquer le code.
//
// Les lettres : A = le héros, X = un mur, P = une porte.
// ALPHABET[0] : le A ALPHABET[23] : le X ALPHABET[15] : le P
// ALPHABET[n] : la n-ième lettre, en comptant à partir de 0 (pas de 1) :
// A = 0, B = 1, C = 2, D = 3, E = 4... L = 11... P = 15, S = 18... X = 23.
// La tuile 0, c'est une case VIDE (un espace).
// La tuile 27, c'est le chiffre 0 ; 28 le 1, 29 le 2, 30 le 3 : 27 + n, le chiffre n.
//
// Le monde : QUATRE salles, deux de large, deux de haut (le 0.95).
//
//      sx = 0      sx = 1
// +-----+-----+
// | salle 0 | salle 1 | sy = 0
// +-----+-----+
// | salle 2 | salle 3 | sy = 1
// +-----+-----+
//
// salle = sx + sy + sy (0 + 0 + 0 = 0, 1 + 0 + 0 = 1, 0 + 1 + 1 = 2, 1 + 1 + 1 = 3)
//
// NOUVEAU : LE MONDE FAIT LE TOUR, dans les deux sens.
// Chaque salle a une sortie de chaque côté : à gauche, à droite, en haut,
// en bas. À droite de la salle 1, on revient à la salle 0 ; à gauche de la
// salle 0, on arrive à la salle 1. En bas de la salle 2, on revient à la
// salle 0 ; en haut de la salle 0, on arrive à la salle 2. Ainsi, CHAQUE
// sortie mène à une salle, et l'écran glisse à chaque fois.
// Avec deux colonnes de salles, changer de colonne, c'est : sx = 1 - sx
// 1 - 0 = 1 (on passe de la colonne 0 à la colonne 1),
// 1 - 1 = 0 (on passe de la colonne 1 à la colonne 0).
// Et changer de ligne de salles : sy = 1 - sy.
//
// NOUVEAU : LE DÉCOR EST PLUS GRAND QUE L'ÉCRAN.
// Le décor de la console fait 32 colonnes sur 32 lignes (256 × 256 pixels) ;
// l'écran n'en montre que 20 colonnes sur 18 lignes (160 × 144 pixels).
// defiler(x, y) choisit QUEL morceau on regarde : x et y sont la place de
// la « caméra », en pixels (8 pixels = 1 case).
//
// colonnes du décor : 0 ..... 19 20 ..... 31
// [ l'écran, camx = 0 ][ caché ]
// (et pareil en hauteur : les lignes 0 à 17 à l'écran, 18 à 31 cachées)
//
// Le décor est un RUBAN, dans les deux sens : après la colonne 31 revient
// la colonne 0 ; après la ligne 31 revient la ligne 0.
//
// La salle, elle, garde ses colonnes 0 à 19 et ses lignes 0 à 17 :
// les lignes 0 à 16 pour le terrain, la ligne 17 pour « SALLE n ».
// bord dit dans quelle colonne DU DÉCOR est sa colonne 0 ;
// haut dit dans quelle ligne DU DÉCOR est sa ligne 0 :
// colonne du décor = (colonne de la salle + bord) % 32
// ligne du décor = (ligne de la salle + haut) % 32
// % 32 : le reste de la division par 32 ; il fait « le tour du ruban ».
// Exemple, bord = 12 : la colonne 19 de la salle est (19 + 12) % 32 = 31,
// et s'il y avait une colonne 20 : (20 + 12) % 32 = 32 % 32 = 0.
```

```
// La caméra regarde toujours la salle : camx = bord × 8, camy = haut × 8.

// ---- Les variables ----
// uint8_t : une boîte qui garde un nombre de 0 à 255, jamais en dessous de 0.

#include <poser>      // pose une tuile sur une case du fond
#include <lire>       // lit la tuile posée sur une case
#include <ALPHABET>   // les lettres de la police : ALPHABET[0] est le A
#include <effacer>    // efface des cases, ou tout le fond
#include <defiler>    // fait glisser tout le fond
#include <chaque>     // répond 1 toutes les n millisecondes
#include <bouton>     // lit un bouton de la manette
```

2. UNE VARIABLE — AX

```
uint8_t ax = 9;          // le A, le héros : sa colonne DANS LA SALLE (0 à 19)...
```

3. UNE VARIABLE — AY

```
uint8_t ay = 8;          // ...et sa ligne (0 à 16). Il part de (9, 8).
```

4. UNE VARIABLE — NX

```
uint8_t nx = 0;          // la case d'ARRIVÉE, calculée AVANT de bouger (le 0.82)
```

5. UNE VARIABLE — NY

```
uint8_t ny = 0;
```

6. UNE VARIABLE — SX

```
uint8_t sx = 0;          // la salle : sa colonne dans le monde (0 ou 1)...
```

7. UNE VARIABLE — SY

```
uint8_t sy = 0;          // ...et sa ligne (0 ou 1). On part de la salle 0.
```

8. UNE VARIABLE — SALLE

```
uint8_t salle = 0;        // son numéro, 0 à 3
```

9. UNE VARIABLE — BORD

```
uint8_t bord = 0;         // NOUVEAU : la colonne du décor où commence la salle (0 à 31)
```

10. UNE VARIABLE — HAUT

```
uint8_t haut = 0;         // NOUVEAU : la ligne du décor où commence la salle (0 à 31)
```

11. UNE VARIABLE — CAMX

```
uint8_t camx = 0;         // NOUVEAU : la place de la caméra, en pixels : de gauche à droite...
```

12. UNE VARIABLE — CAMY

```
uint8_t camy = 0;         // ...et de haut en bas. defiler(camx, camy).
```

13. UNE VARIABLE — SEULE

```
uint8_t seule = 255;       // NOUVEAU : 255 = toutes les colonnes ; 0 à 19 = CETTE colonne seulement
```

14. UNE VARIABLE — SEULEL

```
uint8_t seuleL = 255;    // NOUVEAU : 255 = toutes les lignes ;   0 à 17 = CETTE ligne seulement

// ---- NOUVEAU : poser une lettre sur une case DE LA SALLE ----
// Au 0.95, on écrivait poser(c, l, ...) : la salle était toujours dans les
// colonnes 0 à 19 et les lignes 0 à 17 du décor. Maintenant, elle peut être
// ailleurs : on ajoute bord et haut, et % 32 fait le tour du ruban.
// Et si seule (ou seuleL) n'est pas 255, on ne pose QUE dans cette colonne
// (ou cette ligne) : le reste n'est pas touché (c'est pour le glissement).
// && : « et ». Il faut que les DEUX conditions soient vraies.
```

15. UNE FONCTION — POSERSALLE()

```
void poserSalle(uint8_t c, uint8_t l, uint8_t t) {
    if ((seule == 255 || c == seule) && (seuleL == 255 || l == seuleL)) {
        poser((c + bord) % 32, (l + haut) % 32, t); // t : une lettre, un chiffre, ou 0 pour vide
    }
}

// ---- NOUVEAU : lire la lettre d'une case DE LA SALLE ----
// Même calcul que poserSalle(). « uint8_t » devant le nom : cette fonction
// REND un nombre (la tuile lue). « return » dit lequel.
```

16. UNE FONCTION — LIRESALLE()

```
uint8_t lireSalle(uint8_t c, uint8_t l) {
    return lire((c + bord) % 32, (l + haut) % 32);
}

// ---- n murs X à la suite, vers la DROITE, à partir de (c, l) (le 0.95) ----
// Exemple : mur(4, 4, 3) pose un X en (4, 4), (5, 4) et (6, 4).
// Seul changement : poserSalle() au lieu de poser().
```

17. UNE FONCTION — MUR()

```
void mur(uint8_t c, uint8_t l, uint8_t n) {
    for (uint8_t i = 0; i < n; i++) { // i vaut 0, puis 1, puis 2... jusqu'à n - 1
        poserSalle(c + i, l, ALPHABET[23]); // un X, i cases à droite du départ
    }
}

// ---- n murs X à la suite, vers le BAS, à partir de (c, l) (le 0.95) ----
// Exemple : murDebout(6, 3, 3) pose un X en (6, 3), (6, 4) et (6, 5).
```

18. UNE FONCTION — MURDEBOUT()

```
void murDebout(uint8_t c, uint8_t l, uint8_t n) {
    for (uint8_t i = 0; i < n; i++) {
        poserSalle(c, l + i, ALPHABET[23]); // c'est la LIGNE qui avance, pas la colonne
    }
}

// ---- vider TOUT le décor ----
// Les 32 lignes, sur leurs 32 colonnes (et plus 18 × 20 comme au 0.95) :
// les parties cachées doivent être vides elles aussi, car le glissement
// va les montrer.
```

19. UNE FONCTION — VIDEREcran()

```
void viderEcran() {
    for (uint8_t l = 0; l < 32; l++) {    // l vaut 0, 1, 2... jusqu'à 31
        effacer(0, l, 32);                // 32 cases à partir de la colonne 0 : toute la ligne du décor
    }
}

// ---- NOUVEAU : toute la salle (sx, sy) : murs, portes, et « SALLE n » ----
// C'était le milieu de dessinerSalle() au 0.95. On le met à part pour
// pouvoir l'appeler de plusieurs façons :
//   seule = 255, seuleL = 255 : toute la salle est dessinée ;
//   seule = 7                  : seule la colonne 7 est dessinée ;
//   seuleL = 5                  : seule la ligne 5 est dessinée.
// C'est poserSalle() qui trie : les autres cases sont ignorées.
// Les ouvertures s'écrivent poserSalle(..., 0) : poser une case vide, c'est
// effacer. (effacer() ne sait pas qu'il faut ajouter bord et haut.)
```

20. UNE FONCTION — DESSINERMURS()

```
void dessinerMurs() {
    salle = sx + sy + sy;                // le numéro de la salle

    // 1. le cadre, en entier : les lignes 0 et 16, les colonnes 0 et 19
    mur(0, 0, 20);                        // en haut : de (0, 0) à (19, 0)
    mur(0, 16, 20);                      // en bas : de (0, 16) à (19, 16)
    murDebout(0, 0, 17);                  // à gauche : de (0, 0) à (0, 16)
    murDebout(19, 0, 17);                 // à droite : de (19, 0) à (19, 16)

    // 2. NOUVEAU : les OUVERTURES, de chaque côté, TOUJOURS (le monde fait le tour).
    // Plus besoin de « if » : chaque salle a ses quatre sorties.
    poserSalle(0, 7, 0);                  // à gauche : les lignes 7, 8, 9 de la colonne 0
    poserSalle(0, 8, 0);
    poserSalle(0, 9, 0);
    poserSalle(19, 7, 0);                  // à droite : les lignes 7, 8, 9 de la colonne 19
    poserSalle(19, 8, 0);
    poserSalle(19, 9, 0);
    poserSalle(9, 0, 0);                  // en haut : les colonnes 9 et 10 de la ligne 0
    poserSalle(10, 0, 0);
    poserSalle(9, 16, 0);                 // en bas : les colonnes 9 et 10 de la ligne 16
    poserSalle(10, 16, 0);

    // 3. les murs DANS la salle : chaque salle a les siens (le 0.95)
    if (salle == 0) {
        mur(4, 4, 12);                   // une barre en haut : de (4, 4) à (15, 4)
        poserSalle(3, 12, ALPHABET[15]); // la PORTE de la salle 0, en (3, 12)
    }
    if (salle == 1) {
        murDebout(6, 3, 11);              // une colonne de (6, 3) à (6, 13)...
        murDebout(13, 1, 11);             // ...une autre de (13, 1) à (13, 11)
    }
    if (salle == 2) {
        mur(3, 5, 6);                    // une barre de (3, 5) à (8, 5)...
        mur(11, 11, 6);                  // ...et une autre de (11, 11) à (16, 11)
    }
    if (salle == 3) {
        mur(3, 8, 12);                   // une barre au milieu : de (3, 8) à (14, 8)
        poserSalle(15, 4, ALPHABET[15]); // la PORTE de la salle 3, en (15, 4)
    }

    // 4. NOUVEAU : « SALLE » et son numéro, sur la ligne 17, lettre par lettre.
    // texte() et nombre() ne savent pas ajouter bord et haut ; poserSalle(), si.
    // Et comme ça, la ligne 17 glisse avec la salle.
    poserSalle(0, 17, ALPHABET[18]);     // S
    poserSalle(1, 17, ALPHABET[0]);      // A
    poserSalle(2, 17, ALPHABET[11]);    // L
    poserSalle(3, 17, ALPHABET[11]);    // L
    poserSalle(4, 17, ALPHABET[4]);      // E
    poserSalle(6, 17, 27 + salle);       // le chiffre : 27 + 0 = le 0, 27 + 3 = le 3
}

// ---- dessiner la salle D'UN COUP (le 0.95) ----
// Au départ, et pour les portes. On remet tout à zéro : la salle revient
// dans les colonnes 0 à 19 et les lignes 0 à 17 du décor, la caméra aussi.
```


21. UNE FONCTION — DESSINERSALLE()

```
void dessinerSalle() {
    viderEcran();                // le décor vide, en entier
    bord = 0;                   // la salle commence à la colonne 0...
    haut = 0;                   // ...et à la ligne 0 du décor
    camx = 0;                   // la caméra la regarde depuis le coin (0, 0)
    camy = 0;
    defiler(camx, camy);
    seule = 255;                // toute la salle
    seuleL = 255;
    dessinerMurs();
}

// ---- NOUVEAU : UNE colonne de la nouvelle salle ----
// D'abord, on la vide, lignes 0 à 17 : l'ancienne salle y avait peut-être
// un X, que la nouvelle n'a pas. Puis on y dessine la colonne c, et elle seule.
```

22. UNE FONCTION — COLONNESALLE()

```
void colonneSalle(uint8_t c) {
    seule = c;                  // on ne touche QUE la colonne c
    for (uint8_t l = 0; l < 18; l++) {
        poserSalle(c, l, 0);   // vider, du haut (0) au bas (17)
    }
    dessinerMurs();             // puis dessiner : seule la colonne c est posée
    seule = 255;                // de nouveau toutes les colonnes
}

// ---- NOUVEAU : UNE ligne de la nouvelle salle ----
// La même chose, couchée : on vide la ligne l, colonnes 0 à 19, puis on la dessine.
```

23. UNE FONCTION — LIGNESALLE()

```
void ligneSalle(uint8_t l) {
    seuleL = l;                 // on ne touche QUE la ligne l
    for (uint8_t c = 0; c < 20; c++) {
        poserSalle(c, l, 0);   // vider, de la gauche (0) à la droite (19)
    }
    dessinerMurs();
    seuleL = 255;
}

// ---- NOUVEAU : le GLISSEMENT quand le A sort à GAUCHE ----
//
// La nouvelle salle se met 20 colonnes à GAUCHE de l'ancienne :
// bord - 20, sur le ruban de 32, c'est bord + 12 (32 - 20 = 12).
// Exemple : bord = 0 → la nouvelle salle commence en colonne 12 du décor.
// La caméra RECULE (camx diminue) : l'ancienne salle part vers la droite.
// Juste avant chaque pas, on dessine la colonne qui va entrer par la GAUCHE :
// la 19, puis la 18... jusqu'à la 0.
// tour 0 : la colonne 19 de la salle, dans la colonne (19 + 12) % 32 = 31 du décor, cachée
// tour 11 : la colonne 8, dans la colonne 20 (la dernière colonne cachée)
// tour 12 : la colonne 7, dans la colonne 19. C'était l'ancienne salle, mais
// elle vient de sortir de l'écran par la droite : on peut la remplacer.
// tour 19 : la colonne 0, dans la colonne 12. Fini : camx = 96, l'écran montre 12 à 31.
// Chaque colonne entre en 4 PETITS PAS de 2 pixels, un par image (4 × 2 = 8 pixels) :
// 20 colonnes × 4 images = 80 images, un peu plus d'une seconde.
```

24. UNE FONCTION — GLISSERAGAUCHE()

```
void glisserAGauche() {
    sx = 1 - sx; // l'autre colonne du monde (0 → 1, 1 → 0)
    bord = (bord + 12) % 32; // la nouvelle salle : 20 colonnes plus à gauche
    for (uint8_t k = 0; k < 20; k++) { // 20 tours : une colonne par tour
        colonneSalle(19 - k); // la colonne qui va entrer à gauche : 19, 18... 0
        for (uint8_t p = 0; p < 4; p++) { // 4 petits pas de 2 pixels (p vaut 0, 1, 2, 3)
            image(); // un petit pas par image
            camx = camx - 2; // la caméra RECULE : 0, 254, 252... (0 - 2 fait le tour : 254)
            defiler(camx, camy); // le décor glisse vers la DROITE
        }
    }
}

// ---- NOUVEAU : le GLISSEMENT quand le A sort à DROITE ----
// Le même, dans l'autre sens.
// La nouvelle salle se met 20 colonnes à DROITE de l'ancienne : bord + 20.
// Exemple : bord = 0 → la nouvelle salle commence en colonne 20 du décor
// (les colonnes 20 à 31, puis 0 à 7 : le ruban fait le tour).
// La caméra AVANCE (camx augmente) : l'ancienne salle part vers la gauche.
// On dessine la colonne qui va entrer par la DROITE : la 0, puis la 1... jusqu'à la 19.
// tour 0 : la colonne 0 de la salle, dans la colonne (0 + 20) % 32 = 20 du décor, cachée
// tour 12 : la colonne 12, dans la colonne (12 + 20) % 32 = 0. C'était l'ancienne
// salle, mais elle vient de sortir de l'écran par la gauche.
```

25. UNE FONCTION — GLISSERADROITE()

```
void glisserADroite() {
    sx = 1 - sx; // l'autre colonne du monde
    bord = (bord + 20) % 32; // la nouvelle salle : 20 colonnes plus à droite
    for (uint8_t k = 0; k < 20; k++) { // la colonne qui va entrer à droite : 0, 1... 19
        colonneSalle(k);
        for (uint8_t p = 0; p < 4; p++) {
            image();
            camx = camx + 2; // la caméra AVANCE : 0, 2, 4... (254 + 2 fait le tour : 0)
            defiler(camx, camy); // le décor glisse vers la GAUCHE
        }
    }
}

// ---- NOUVEAU : le GLISSEMENT quand le A sort en HAUT ----
// Pareil, mais debout : on travaille ligne par ligne, et c'est camy qui bouge.
// La salle a 18 lignes (0 à 17) ; le décor en a 32.
// La nouvelle salle se met 18 lignes AU-DESSUS : haut - 18, sur le ruban de 32,
// c'est haut + 14 (32 - 18 = 14).
// On dessine la ligne qui va entrer par le HAUT : la 17, puis la 16... jusqu'à la 0.
// 18 lignes × 4 images = 72 images.
```

26. UNE FONCTION — GLISSERENHAUT()

```
void glisserEnHaut() {
    sy = 1 - sy;                                // l'autre ligne du monde (0 → 1, 1 → 0)
    haut = (haut + 14) % 32;                     // la nouvelle salle : 18 lignes plus haut
    for (uint8_t k = 0; k < 18; k++) {          // 18 tours : une ligne par tour
        ligneSalle(17 - k);                     // la ligne qui va entrer en haut : 17, 16... 0
        for (uint8_t p = 0; p < 4; p++) {
            image();
            camy = camy - 2;                     // la caméra MONTE : le décor glisse vers le BAS
            defiler(camx, camy);
        }
    }
}

// ---- NOUVEAU : le GLISSEMENT quand le A sort en BAS ----
// La nouvelle salle se met 18 lignes AU-DESSOUS : haut + 18.
// On dessine la ligne qui va entrer par le BAS : la 0, puis la 1... jusqu'à la 17.
```

27. UNE FONCTION — GLISSERENBAS()

```
void glisserEnBas() {
    sy = 1 - sy;
    haut = (haut + 18) % 32;                     // la nouvelle salle : 18 lignes plus bas
    for (uint8_t k = 0; k < 18; k++) {          // la ligne qui va entrer en bas : 0, 1... 17
        ligneSalle(k);
        for (uint8_t p = 0; p < 4; p++) {
            image();
            camy = camy + 2;                     // la caméra DESCEND : le décor glisse vers le HAUT
            defiler(camx, camy);
        }
    }
}

// ---- main() : c'est ICI que la console commence ----
```

28. LE POINT D'ENTRÉE : C'EST ICI QUE LA CONSOLE ARRIVE

```
int main() {
    dessinerSalle();
    poserSalle(ax, ay, ALPHABET[0]);

    while (true) {
        image();

        // ---- le A bouge avec la croix, un pas toutes les 150 ms (le 0.93.1) ----
        if (chaque(150)) {
            // 1. la case d'ARRIVÉE (le 0.82) ; une seule flèche à la fois (else if)
            nx = ax;
            ny = ay;
            if (bouton(DROITE)) {
                nx = ax + 1;
            } else if (bouton(GAUCHE)) {
                nx = ax - 1;
            } else if (bouton(BAS)) {
                ny = ay + 1;
            } else if (bouton(HAUT)) {
                ny = ay - 1;
            }

            // 2. une flèche est tenue : l'arrivée n'est pas la case où l'on est (!= : « différent de »)
            if (nx != ax || ny != ay) {
                // 3. NOUVEAU : l'arrivée est HORS de la salle : l'écran GLISSE vers la nouvelle.
                // Ces tests viennent en premier : lireSalle(255, 8) n'aurait pas de sens.
                // Avant de glisser, on retire le A de l'ancienne salle (sinon, il
                // partirait avec elle) ; après, on le pose, du côté par où il entre.
                if (nx == 255) {
                    poserSalle(ax, ay, 0);
                    glisserAGauche();
                    ax = 19;
                    poserSalle(ax, ay, ALPHABET[0]);
                } else if (nx == 20) {
                    poserSalle(ax, ay, 0);
                    glisserADroite();
                    ax = 0;
                    poserSalle(ax, ay, ALPHABET[0]);
                } else if (ny == 255) {
                    poserSalle(ax, ay, 0);
                    glisserEnHaut();
                    ay = 16;
                    poserSalle(ax, ay, ALPHABET[0]);
                } else if (ny == 17) {
                    poserSalle(ax, ay, 0);
                    glisserEnBas();
                    ay = 0;
                    poserSalle(ax, ay, ALPHABET[0]);
                } else if (lireSalle(nx, ny) == ALPHABET[15]) {
                    // 4. ...l'arrivée est une PORTE P : elle mène à l'autre porte (le 0.95)
                    if (salle == 0) {
                        sx = 1;
                        sy = 1;
                        ax = 14;
                        ay = 4;
                    } else {
                        sx = 0;
                        sy = 0;
                        ax = 4;
                        ay = 12;
                    }
                    dessinerSalle();
                    poserSalle(ax, ay, ALPHABET[0]);
                } else if (lireSalle(nx, ny) == 0) {

```

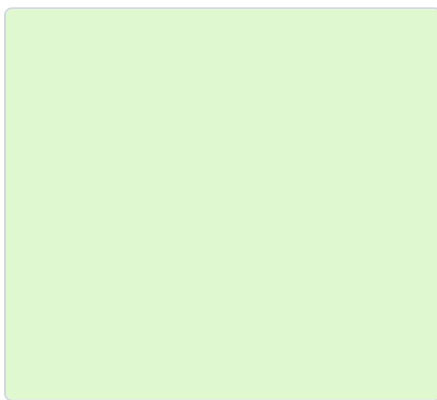
```

// 5. ...l'arrivée est VIDE : le A y va.
poserSalle(ax, ay, 0);           // le A quitte sa case (on y pose du vide)...
ax = nx;                         // ...prend la case d'arrivée...
ay = ny;
poserSalle(ax, ay, ALPHABET[0]); // ...et y est dessiné
}
// 6. ...sinon, l'arrivée est un X : il ne se passe rien. C'est ça, un mur.
}
}
}
}

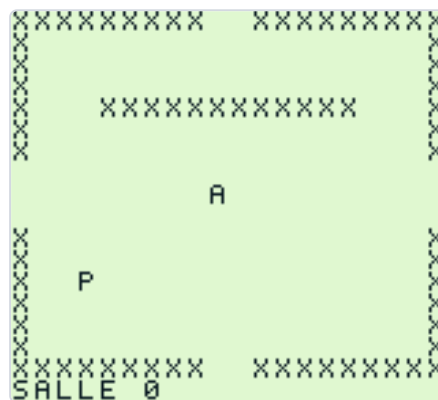
```

3. Ce qui se passe, étape par étape

La cartouche a été compilée et exécutée pour de bon ; voici son écran aux instants qui comptent. Une image identique à la précédente n'est pas montrée — un programme qui ne bouge pas n'a qu'une seule image, et c'est la vérité sur ce programme.



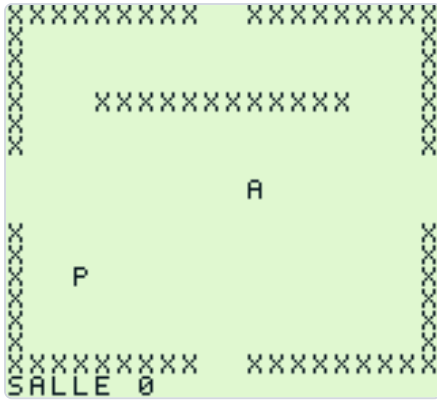
la toute première image — la console vient de démarrer — image 1



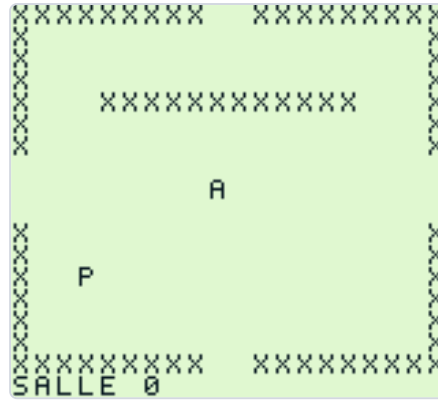
un tiers de seconde plus tard : le premier dessin est posé — image 20

4. Ce que les touches changent

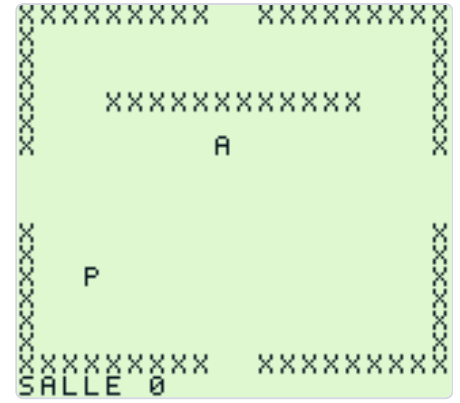
Chaque bouton que le programme lit a été enfoncé douze images, puis relâché. Voici l'écran juste après.



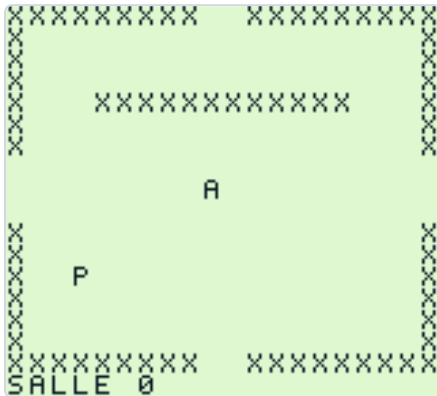
DROITE — l'écran a changé



GAUCHE — l'écran a changé



HAUT — l'écran a changé



BAS — l'écran a changé

5. Ce qu'on doit voir

À chaque sortie, l'écran glisse dans le sens du A : à gauche, à droite, en haut, en bas. Les portes P téléportent d'un coup.

6. Ce que la console répond

Ces contrôles ne sont pas décoratifs : ce sont ceux que `verifier-tuto.mjs` exige de cette leçon. Ils viennent d'être joués, et voici ce qu'ils ont mesuré.

- ✓ sorti à GAUCHE : l'écran glisse, 2 pixels par image, vers la salle 1 (80 places de caméra)
- ✓ sorti à DROITE : il glisse dans l'autre sens, retour à la salle 0 (80 places)
- ✓ sorti en BAS : il glisse vers le haut, la salle 2 arrive par le bas (72 places)
- ✓ sorti en HAUT : il glisse vers le bas, retour à la salle 0 (72 places)

Ensuite : **0.96. Une lettre qui tire — un tir, tout seul** — chapitre 0 : Avant tout. · gameboy3 — écrire une cartouche Game Boy en C++